

Design Snake & Ladder

Amazon • Oracle • Zoho • service MNCs – "ఇది పిల్లల ఆట, పది నిమిషాల్లో అయిపోతుంది" అని అందరూ అనుకుంటారు.

మూడు విరుపులు. మొదటిది ఆటగాడిని **పాము నోట్లో నిలబెడుతుంది**. రెండోది అతన్ని **103వ గడిలో** పెడుతుంది. మూడోది – మొదటిదాన్ని సరిచేస్తే – **ఆటని అనంతంగా తిప్పుతుంది**.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output నిజంగా `node lo run చేసినదే` – 50,000 ఆటల అనుకరణతో సహా.

నేను దీన్ని తక్కువ అంచనా వేశాను – మరియు తప్పు అని తేలింది

ఈ series రాస్తున్నప్పుడు నేను Snake & Ladder ని "పలచని problem" అని అనుకున్నాను. Board, పాచిక, ఆటగాళ్ళు – వ్యూహమే లేదు, కాబట్టి design lo ఏముంటుంది?

Code రాసి చూశాక మూడు నిజమైన విరుపులు దొరికాయి, మరియు అందులో ఒకటి అనంత లూప్. మరియు చివరికి ఒక కొలత నా అంచనాని పూర్తిగా తలకిందులు చేసింది (§13).

ఇది కూడా ఒక పాఠమే: ఒక problem పలచనిదా కాదా అనేది – దాన్ని రాసి చూసేదాకా తెలియదు.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు వ్యూహం లేకపోతే design ఏముంది?
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: ఒక్క జంప్

3. Step – `from + die`, ఆపై ఒక Map చూడటం
4. మొదటి విరుపు – నిచ్చిన ఎక్కి పాము నోట్లో నిలబడటం
5. Step – గొలుసుని ముందే పరిష్కరించడం

Part 3 – రెండో విరుపు: 103వ గడి

6. Step – "100 దాటితే" అంటే ఏమిటి?
7. రెండో విరుపు – board lo లేని గడి
8. Step – ముగింపు నియమం ఒక మారే వస్తువు

Part 4 – మూడో విరుపు: చెల్లని board

9. Step – board ని ఎవరు తనిఖీ చేస్తారు?
10. మూడో విరుపు – వలయం, మరియు ఆట ఆగదు
11. Step – నిర్మాణ సమయంలోనే తిరస్కరించడం

Part 5 – పూర్తి system

12. Step – పాచికని బయటికి తీయడం
13. మొత్తం code · నడిపి చూద్దాం · **50,000 ఆటల కొలత**

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటీతో చెప్పాల్సిన English script
16. Follow-ups – బహుళ ఆటగాళ్ళు, ఆన్‌లైన్, board తయారీ
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు వ్యూహం లేకపోతే design ఏముంది?

"Design the game Snake and Ladder."

ఈ ఆటలో ఆటగాడు ఏ నిర్ణయమూ తీసుకోడు. పాచిక వేయడం, కదలడం – అంతే. మరి design చేయడానికి ఏముంది?

నిర్ణయాలు ఆటగాడివి కావు – అవి నియమాలవి

వ్యూహం లేని ఆట = నియమాలన్నీ system బాధ్యత

1 · జంప్ గొలుసులు

నిచ్చిన ఎక్స్‌క్లెసివ్ అక్కడ
ఒక పాము ఉంటే?
అగాలా, కొనసాగాలా?

→ §4

2 · ముగింపు

97 వద్ద ఉండి 6 వేస్తే?
అగాలా, వెనక్కి తిరగాలా,
గెలవాలా?

→ §7 · 103వ గడి

3 · Board చెల్లుతుందా

పైకి వెళ్ళే పాము?
ఒకే గడి నుంచి రెండు?
ఒక వలయం?

→ §10 · అట అగదు

అందుకే ఇది ఒక మంచి interview problem

Algorithm లేదు, data structure లేదు, వ్యూహం లేదు – కాబట్టి దాచుకోవడానికి ఏమీ లేదు.

మిగిలేది ఒక్కటే: నియమాలని ఎంత జాగ్రత్తగా ఆలోచించారు అనేది.

"ఒక Map lo snakes మరియు ladders పెడతాను" అని 5 నిమిషాల్లో ముగించేవాళ్ళు చాలా మంది.

మిగిలిన 40 నిమిషాలూ interviewer ఆ Map ని విరగొట్టడానికే.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
నిచ్చిన చివర ఒక పాము ఉంటే – దాన్ని కూడా అనుసరించాలా?	ఇదే §4 – మరియు ఇది మీరే అడగాలి
100 దాటితే? (97 + 6)	§8 – మూడు సాధారణ నియమాలు ఉన్నాయి

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
6 వేస్తే మళ్ళీ వంతు వస్తుందా?	ఒక ఎంపిక – <code>extraTurnOnSix</code>
మొదలుపెట్టడానికి 6 కావాలా?	ఒక ఎంపిక; అడగకపోతే వదిలేయండి
Board ఎవరు ఇస్తారు – అది తప్పు అయితే?	§11 – చాలా మంది దీన్ని అస్సలు ఆలోచించరు
ఎంతమంది ఆటగాళ్ళు?	Array ఒకటి చాలు – 2 అని hardcode చేయొద్దు
ఒకే గడిలో ఇద్దరు ఉండొచ్చా?	సాధారణంగా అవును; కాదంటే ఒక కొత్త నియమం

మొదటి ప్రశ్న – మరియు ఎందుకు అది INTERVIEWER కి నచ్చుతుంది

"నేను నిచ్చిన ఎక్కి 22వ గడికి చేరాను. అక్కడ ఒక పాము నోరు ఉంది. నేను 22 వద్దే ఉంటానా, లేక పాము నన్ను మింగుతుందా?"

సరైన జవాబు: **పాము మింగుతుంది.** ఆటగాడు ఆ గడి మీద నిలబడ్డాడు, కాబట్టి ఆ గడి నియమం వర్తిస్తుంది. బోర్డు మీద ఆడినా అదే జరుగుతుంది.

ఈ ప్రశ్న ఎందుకు మంచిది? – ఇది ఒక **నియమ ప్రశ్న**, code ప్రశ్న కాదు. మరియు దీన్ని అడిగితే, మీరు **ఆటని ఊహించుకుంటున్నారని** తెలుస్తుంది, కేవలం classes గేయట్లేదని.

Part 2 – మొదటి విరుపు: ఒక్క జంప్

3. Step – from + die , అపై ఒక Map చూడటం

```
class Game {
  constructor(snakes, ladders, players) {
    this.jumps = new Map([...Object.entries(snakes), ...Object.entries(ladders)]);
    this.pos = new Map(players.map(p => [p, 0]));
    this.players = players;
    this.turn = 0;
  }
  roll(die) {
    const p = this.players[this.turn % this.players.length];
    let next = this.pos.get(p) + die;
    // ఒక జంప్ మాత్రమే
    if (this.jumps.has(String(next))) next = this.jumps.get(String(next));
    this.pos.set(p, next);
    this.turn++;
    return { player: p, to: next, won: next >= 100 };
  }
}
```

పనిచేస్తుంది – పాములు, నిచ్చెనలు, వంతులు, గెలుపు, అన్నీ:

```
{ player: 'ravi', to: 38, won: false }    ← 0 + 1 = 1, నిచ్చెన 1→38
{ player: 'asha', to: 3, won: false }
{ player: 'ravi', to: 46, won: false }
{ player: 'asha', to: 5, won: false }
{ player: 'ravi', to: 50, won: false }
```

పదిహేను పంక్తులు, పూర్తి ఆట. ఇక్కడ చాలా మంది ఆగిపోతారు.

4. మొదటి విరుపు – నిచ్చెన ఎక్కి పాము నోట్లో నిలబడటం

ఒక board పెడదాం: 3 → 22 (నిచ్చెన), మరియు 22 → 8 (పాము).

1 • గొలుసు జంప్లు – నిచ్చెన ఎక్కి పాము నోట్లో పడటం
స్థానం 1 నుంచి 2 వేస్తే → 22
కానీ 22 వద్ద ఒక పాము ఉంది, అది 8 కి దిగుతుంది.
అటుగాడు నిజంగా ఎక్కడ ఉండాలి? → 8

మొదటి విరుపు - ఆటగాడు ఒక పాము నోట్స్ నిలబడి ఉన్నాడు

Code 22 అని చెప్పింది. కానీ 22 అనేది ఒక పాము నోరు - బోర్డు మీద ఎవరూ అక్కడ ఉండలేరు.

మరియు ఇది మరింత చెడ్డది: తర్వాతి వంతులో అతను 22 నుంచి కదులుతాడు, ఆ పాము ఎప్పటికీ వర్తించదు. అంటే ఆ పాము ఈ ఆటలో ఉనికిలోనే లేదు.

మౌలిక తప్పు: if అనేది "ఒకసారి తనిఖీ చెయ్యి" అని చెప్పింది. కానీ నియమం "ఎక్కడ ఆగుతావా అక్కడి నియమం వర్తిస్తుంది" - మరియు అది పునరావృతం. if కి బదులు while కావాలి.

(మరియు ఆ while మూడో విరుపుని తెస్తుంది - §10.)

5. Step - గొలుసుని ముందే పరిష్కరించడం

ఇక్కడ ఒక మంచి ఆలోచన ఉంది: గొలుసులు board యొక్క లక్షణం, ఆట యొక్క లక్షణం కాదు. "22 మీద నిలబడితే చివరికి 8" అనేది ఆట మొదలవకముందే తెలిసిన విషయం.

కాబట్టి దాన్ని ఒకసారి, నిర్మాణ సమయంలో లెక్కిద్దాం:

```
// ఒక గడి మీద నిలబడితే చివరికి ఎక్కడ ఆగుతాం - గొలుసు మొత్తం  
landOn(square) { return this.#resolved.get(square) ?? square; }
```

```
--- గొలుసు జంప్లు --- (board: 3→22, 22→8, 40→75, 75→99)  
3 మీద నిలబడితే → 8  
22 మీద నిలబడితే → 8  
40 మీద నిలబడితే → 99  
75 మీద నిలబడితే → 99  
50 మీద నిలబడితే → 50
```

3 → 8 (రెండు జంప్లు) మరియు 40 → 99 (రెండు నిచ్చెనలు). మరియు 50 - జంప్ లేని గడి - అలాగే ఉంది.

ఎందుకు ఆట సమయంలో కాకుండా నిర్మాణ సమయంలో?

మూడు కారణాలు:

- 1 • ఆట lo landOn ఒక Map lookup - ఒక లూప్ కాదు.
- 2 • గొలుసు ఎప్పుడూ ఒకటే. ప్రతి వంతులోనూ దాన్ని మళ్ళీ నడవడం వృథా.
- 3 • మరియు ఇది ముఖ్యమైనది - ఆ గొలుసులని నడుస్తున్నప్పుడే వలయాలు కనిపిస్తాయి (§11). ఆట మధ్యలో ఒక అనంత లూప్ కనుక్కోవడం కంటే, board కట్టేటప్పుడే దాన్ని తిరస్కరించడం మేలు.

ఇది Deep Dive 12 §5 lo చూసిన అదే ఆలోచన: నిర్మాణాన్ని ఒకసారి లెక్కించి నిల్వ చెయ్యి, ప్రతిసారీ తిరిగి కనుక్కోవద్దు.

Part 3 – రెండో విరుపు: 103వ గడి

6. Step – "100 దాటేత" అంటే ఏమిటి?

ఆటగాడు 97వ గడిలో ఉన్నాడు. 6 వేశాడు. **103**.

కానీ board lo 100 గడులే ఉన్నాయి.

7. రెండో విరుపు – board lo లేని గడి

2 • 100 దాటిపోవడం
స్థానం 97 నుంచి 6 వేస్తే → 103 (గరిష్ఠం 100)

రెండో విరుపు – WON: NEXT >= 100 అనేది ఒక నిర్ణయాన్ని దాచేసింది

§3 యొక్క code `next >= 100` అని చూస్తుంది, కాబట్టి 103 "గెలుపు" అవుతుంది. అది ఒక crash కాదు – కానీ అది ఒక ఆట నియమాన్ని నిశ్శబ్దంగా ఎంచుకుంది.

నిజ ప్రపంచంలో మూడు వేర్వేరు నియమాలు వాడతారు:

- 1 • అక్కడే ఉండిపోవడం – 100 మీద సరిగ్గా పడాలి. దాటేత వంతు వృథా. (అత్యంత సాధారణం)
- 2 • వెనక్కి తిరగడం – 103 అంటే 100 నుంచి 3 వెనక్కి = 97.
- 3 • దాటినా గెలుపే – పిల్లలతో ఆడేటప్పుడు.

మూడూ చెల్లుబాటుయ్యేవే. కాబట్టి ఇది ఒక `if` కాదు – ఇది ఒక **మారే నియమం**, మరియు §13 lo మనం దాని ప్రభావాన్ని కొలుస్తాం.

8. Step – ముగింపు నియమం ఒక మారే వస్తువు

```
class MustLandExactly { // దాటేత అక్కడే ఉండిపోవడం (సాధారణ నియమం)
    apply(from, raw, size) { return raw > size ? from : raw; }
}
class BounceBack { // దాటిన మేరకు వెనక్కి
    apply(from, raw, size) { return raw > size ? size - (raw - size) : raw; }
}
class AnyOvershootWins { // దాటినా గెలుపే
    apply(from, raw, size) { return Math.min(raw, size); }
}
```

--- 100 దాటితే ---

అక్కడే ఉండిపోవడం $96 + 6 \rightarrow 96$

అక్కడే ఉండిపోవడం $99 + 6 \rightarrow 99$

వెనక్కి తిరగడం $96 + 6 \rightarrow 98$

వెనక్కి తిరగడం $99 + 6 \rightarrow 95$

గమనించండి: "వెనక్కి తిరగడం" నియమంలో 99 నుంచి 6 వేస్తే **95** – అంటే ఆటగాడు వెనక్కి వెళ్ళాడు. అది సరైనదే, మరియు ఆ నియమం ఎందుకు ఆటని పొడిగిస్తుందో §13 lo కొలిచి చూస్తాం.

మరియు ఒక సూక్ష్మమైన వివరం: వెనక్కి వచ్చిన గడి మీద **ఒక పాము ఉండొచ్చు** – మరియు అది వర్తించాలి. అందుకే `finish.apply()` తర్వాత `board.landOn()` పిలుస్తాం, ముందు కాదు.

Part 4 – మూడో విరుపు: చెల్లని board

9. Step – board ని ఎవరు తనిఖీ చేస్తారు?

§5 lo గొలుసులని అనుసరించాలని నిర్ణయించాం. దాన్ని సహజంగా రాస్తే:

```
let next = from + die;  
while (jumps.has(next)) next = jumps.get(next);    // గొలుసుని పూర్తిగా అనుసరించడం
```

ఇది §4 విరుపుని సరిచేస్తుంది. కానీ ఒక board ఇలా ఉంటే?

5 → 25 (నిచ్చిన), 25 → 5 (పాము)

10. మూడో విరుపు – వలయం, మరియు ఆట ఆగదు

3 · వలయం – నిచ్చిన పైకి, పాము కిందకి, మళ్ళీ అదే చోటికి
5 → 25 (నిచ్చిన), 25 → 5 (పాము)
x INFINITE_LOOP: 20 జంప్ల తర్వాత కూడా ఆగలేదు

ఒక నిచ్చిన, ఒక పాము, ఒక అనంత లూప్

రెండూ విడివిడిగా సహజమే – కలిసి ఆటని ఆపేస్తాయి



ఆటగాడు 5 మద పడ్డాడు → 25 కి → 5 కి → 25 కి → ...

ఇది ఒక crash కాదు. Error message లేదు. **ఆట కేవలం ఆగిపోతుంది.**

మరియు ఇది **ఒక నిర్దిష్ట ఆటగాడు ఒక నిర్దిష్ట సంఖ్య** వేసినప్పుడు మాత్రమే జరుగుతుంది –

అంటే tests lo కనిపించదు, ఉత్పత్తిలో కనిపిస్తుంది.

మూడో విరుపు - తప్పు CODE LO లేదు, అది డేటాలో ఉంది

`while (jumps.has(next))` అనే code పూర్తిగా సరైనది. Board చెల్లుబాటుయ్యేది అయితే అది ఎప్పుడూ ఆగుతుంది.

సమస్య ఏమిటంటే - ఎవరూ board చెల్లుబాటుయ్యేదా అని అడగలేదు.

మరియు ఇది ఒక్క వలయమే కాదు. ఒక board ఇవి కూడా కలిగి ఉండొచ్చు:

- పైకి వెళ్ళే పాము / కిందకి వెళ్ళే నిచ్చెన - నియమాల ఉల్లంఘన
- ఒక గడి నుంచి రెండు జంప్లు - ఏది వర్తిస్తుంది?
- తనలోకే జంప్ (10 → 10) - అర్థరహితం
- పరిధి బయట (105 → 3, లేదా 50 → 250)
- గడి 100 నుంచి జంప్ - గెలిచాక ఎక్కడికి?

ఇవన్నీ అట సమయంలో కనిపించవు - ఎవరో ఆ గడి మీద పడేవరకు.

11. Step - నిర్మాణ సమయంలోనే తిరస్కరించడం

Board కట్టేటప్పుడే అన్నిటినీ తనిఖీ చేద్దాం - మరియు అన్ని సమస్యలనీ ఒకసారి నివేదిద్దాం:

```
constructor(size = 100, jumps = {}) {
  this.size = size;
  const pairs = Array.isArray(jumps)
    ? jumps : Object.entries(jumps).map(([k, v]) => [+k, v]);
  const problems = [];
  for (const [from, to] of pairs) {
    if (!Number.isInteger(from) || !Number.isInteger(to))
      { problems.push(`NOT_INTEGER: ${from}→${to}`); continue; }
    if (from < 1 || from > size - 1)
      { problems.push(`BAD_START: ${from} (1..${size-1} మాత్రమే)`); continue; }
    if (to < 1 || to > size)
      { problems.push(`BAD_END: ${from}→${to}`); continue; }
    if (to === from) { problems.push(`SELF_JUMP: ${from}`); continue; }
    if (this.#jumps.has(from))
      { problems.push(`DUPLICATE_START: ${from}`); continue; }
    this.#jumps.set(from, to);
  }
  // వలయాలు - ఒక గొలుసు తనలోకే తిరిగి వస్తే board చెల్లదు
  for (const start of this.#jumps.keys()) {
    const seen = new Set([start]);
    let cur = this.#jumps.get(start);
    while (this.#jumps.has(cur)) {
      if (seen.has(cur)) {
        problems.push(`CYCLE: ${[...seen, cur].join('→')}`); break;
      }
      seen.add(cur); cur = this.#jumps.get(cur);
    }
    if (!this.#jumps.has(cur)) this.#resolved.set(start, cur);
  }
  if (problems.length) {
```

```

const e = new Error(`INVALID_BOARD: ${problems.length} సమస్యలు`);
e.problems = problems;
throw e;
}
}

```

```

--- చెల్లని boards ---
వలయం                → INVALID_BOARD: 2 సమస్యలు
    CYCLE: 5→25→5
    CYCLE: 25→5→25
తనలోక                → INVALID_BOARD: 1 సమస్యలు
    SELF_JUMP: 10
ఒకే గడి నుంచి రెండు    → INVALID_BOARD: 1 సమస్యలు
    DUPLICATE_START: 7
పరిధి బయట            → INVALID_BOARD: 3 సమస్యలు
    BAD_START: 0 (1..99 మాత్రమే)
    BAD_END: 50→250
    BAD_START: 105 (1..99 మాత్రమే)

```

మూడు DESIGN వివరాలు - ప్రతిదీ ఉద్దేశపూర్వకం

1 • అన్ని సమస్యలనీ సేకరించడం, మొదటిదానికే ఆగిపోకపోవడం. చివరి ఉదాహరణలో మూడు సమస్యలు చెప్పింది. ఒక board file సరిచేసేవారికి - ఒక్కొక్కటిగా కనుక్కోవడం కంటే ఇది ఎంతో మేలు.

2 • ఇక్కడ throw, {ok:false} కాదు. ఈ series lo చాలాసార్లు తిరస్కరణని ఒక విలువగా ఇచ్చాం. కానీ ఇది వేరు - ఒక చెల్లని board తో ఆట ఉనికిలో ఉండకూడదు. Constructor ఒక విలువ తిరిగి ఇవ్వలేదు, మరియు "సగం కట్టిన board" అనేది ప్రమాదకరం.

3 • jumps ఒక array కూడా కావచ్చు. JavaScript object lo నకిలీ keys ఉండలేవు - {7:20, 7:44} అనేది ఒకే entry. కానీ ఒక board file lo నకిలీలు సులభంగా వస్తాయి. అందుకే [[7,20],[7,44]] రూపాన్ని కూడా అంగీకరిస్తున్నాం - లేకపోతే ఆ తనిఖీకి అర్థమే లేదు.

Part 5 – పూర్తి system

12. Step – పాచికని బయటికి తీయడం

ఒక చివరి సమస్య: `Math.random()` ని ఎలా పరీక్షించాలి?

```
class Die {
  constructor(sides = 6, rand = Math.random) {
    this.sides = sides; this.rand = rand;
  }
  roll() { return 1 + Math.floor(this.rand() * this.sides); }
}
class ScriptedDie { // పరీక్షలకి
  constructor(values) { this.values = [...values]; this.i = 0; }
  roll() { return this.values[this.i++ % this.values.length]; }
}
```

ఇది ఒక చిన్న CLASS – కానీ అది మూడు పనులు చేస్తుంది

1 • పరీక్షించగలగడం. `ScriptedDie([1,3,8,...])` తో ఒక ఆట ప్రతీసారీ ఒకేలా నడుస్తుంది. లేకపోతే "6 వేస్తే ఏమవుతుంది?" అని పరీక్షించడానికి `Math.random` ని mock చేయాలి.

2 • అనుకరణ. `new Die(6, seededRand)` – §13 lo 50,000 ఆటలని పునరుత్పత్తి చేయగలిగేలా నడపడానికి ఇదే కావాలి.

3 • మార్పులు. 12-ముఖాల పాచిక? రెండు పాచికలు? – `Die` ని మార్చితే చాలు, `Game` ని కాదు.

Deep Dive 14 §12 lo సమయాన్ని ఇలాగే బయటికి తీశాం. **నియమం ఒకటే: system బయటి ప్రపంచం నుంచి ఏదైనా అడిగితే, దాన్ని బదులుగా ఇవ్వాలి.**

13. మొత్తం code · నడిపి చూద్దాం · 50,000 ఆటల కొలత

నిర్మాణం · ఐదు ముక్కలు

Game కి నియమాలు తెలియవు - అవి లోపలికి ఇవ్వబడతాయి



landOn ఎప్పుడూ ఆగుతుంది. అంటే Game లో ఒక్క రక్షణ తనిఖీ కూడా అవసరం లేదు.

```
'use strict';
class Board {
  #jumps = new Map(); // from → to
  #resolved = new Map(); // from → గొలుసు చివర
  // jumps: { from: to } లేదా [[from, to], ...] - రెండో రూపం నకిలీలని చూపగలదు
  constructor(size = 100, jumps = {}) {
    this.size = size;
    const pairs = Array.isArray(jumps)
      ? jumps : Object.entries(jumps).map(([k, v]) => [+k, v]);
    const problems = [];
    for (const [from, to] of pairs) {
      if (!Number.isInteger(from) || !Number.isInteger(to))
        { problems.push(`NOT_INTEGER: ${from}→${to}`); continue; }
      if (from < 1 || from > size - 1)
        { problems.push(`BAD_START: ${from} (1..${size-1} మాత్రమే)`); continue; }
      if (to < 1 || to > size)
        { problems.push(`BAD_END: ${from}→${to}`); continue; }
      if (to === from) { problems.push(`SELF_JUMP: ${from}`); continue; }
      if (this.#jumps.has(from))
        { problems.push(`DUPLICATE_START: ${from}`); continue; }
      this.#jumps.set(from, to);
    }
    for (const start of this.#jumps.keys()) {
      const seen = new Set([start]);
      let cur = this.#jumps.get(start);
      while (this.#jumps.has(cur)) {
        if (seen.has(cur)) {
          problems.push(`CYCLE: ${[...seen, cur].join('→')}`); break;
        }
        seen.add(cur); cur = this.#jumps.get(cur);
      }
      if (!this.#jumps.has(cur)) this.#resolved.set(start, cur);
    }
    if (problems.length) {
      const e = new Error(`INVALID_BOARD: ${problems.length} సమస్యలు`);
      e.problems = problems;
      throw e;
    }
  }
}
```

```

    get jumps() { return new Map(this.#jumps); }
    landOn(square) { return this.#resolved.get(square) ?? square; }
    kind(from) {
        const to = this.#jumps.get(from);
        return to === undefined ? null : (to > from ? 'LADDER' : 'SNAKE');
    }
}

class MustLandExactly { // దాటితే అక్కడే ఉండిపోవడం
    apply(from, raw, size) { return raw > size ? from : raw; }
}

class BounceBack { // దాటిన మేరకు వెనక్కి
    apply(from, raw, size) { return raw > size ? size - (raw - size) : raw; }
}

class AnyOvershootWins { // దాటినా గెలుపే
    apply(from, raw, size) { return Math.min(raw, size); }
}

class Die {
    constructor(sides = 6, rand = Math.random) {
        this.sides = sides; this.rand = rand;
    }
    roll() { return 1 + Math.floor(this.rand() * this.sides); }
}

class ScriptedDie {
    constructor(values) { this.values = [...values]; this.i = 0; }
    roll() { return this.values[this.i++ % this.values.length]; }
}

class Game {
    #history = [];
    constructor(board, players, {
        die = new Die(), finish = new MustLandExactly(), extraTurnOnSix = false,
    } = {}) {
        if (players.length < 1) throw new Error('NO_PLAYERS');
        Object.assign(this, { board, players, die, finish, extraTurnOnSix });
        this.pos = new Map(players.map(p => [p, 0]));
        this.turnIndex = 0;
        this.winner = null;
    }
    get turn() { return this.players[this.turnIndex]; }
    get isOver() { return this.winner !== null; }

    play(by = this.turn) {
        if (this.isOver) return { ok: false, reason: 'GAME_OVER', winner: this.winner };
        if (by !== this.turn)
            return { ok: false, reason: `NOT_YOUR_TURN: ${this.turn}` };

        const die = this.die.roll();
        const from = this.pos.get(by);
        const raw = from + die;
        const capped = this.finish.apply(from, raw, this.board.size);
        const landed = this.board.landOn(capped); // గొలుసు మొత్తం అనుసరించడం

        this.pos.set(by, landed);
        const jumped = landed !== capped ? this.board.kind(capped) : null;
        this.#history.push({ by, die, from, capped, landed, jumped });

        if (landed === this.board.size) {

```

```

    this.winner = by;
    return { ok: true, die, from, to: landed, jumped, won: true };
  }
  if (!(this.extraTurnOnSix && die === this.die.sides))
    this.turnIndex = (this.turnIndex + 1) % this.players.length;
  return { ok: true, die, from, to: landed, jumped, next: this.turn };
}
get history() { return [...this.#history]; }
}

```

--- ఒక పూర్తి ఆట (scripted dice) ---

```

ravi 1 → 0 → 38 (LADDER)
asha 3 → 0 → 3
ravi 8 → 38 → 46
asha 2 → 3 → 5
ravi 4 → 46 → 50
asha 6 → 5 → 11
ravi 5 → 50 → 55
asha 2 → 11 → 13
ravi 6 → 55 → 61
asha 4 → 13 → 7 (SNAKE)
ravi 3 → 61 → 60 (SNAKE)
asha 5 → 7 → 12

```

--- తిరస్కరణలు ---

```

తప్పు వంతు : { ok: false, reason: 'NOT_YOUR_TURN: ravi' }

```

మరియు ఇప్పుడు – ఒక కొలత నా అంచనాని తలకిందులు చేసింది

Die ని బయటికి తీశాం కాబట్టి (§12), ఒక seeded RNG ఇచ్చి **50,000** ఆటలని పునరుత్పత్తి చేయగలిగేలా నడపాచ్చు:

50,000 ఆటలు · ఒక్క ఆటగాడు · 6-ముఖాల పాచిక

board	సగటు	మధ్యస్థ	p90	p99	గరిష్ఠం
పాములు/నిచ్చెనలు లేకుండా	33.3	32	41	53	84
classic board	33.4	29	58	101	250
నిచ్చెనలు మాత్రమే	21.8	21	32	45	75
పాములు మాత్రమే	90.3	72	169	307	641

పాములు మరియు నిచ్చెనలు ఆట పోడవుని మార్పు - అవి అనిశ్చితని మారుస్తాయి

మొదటి రెండు వరుసలు చూడండి. **సగటు దాదాపు ఒకటే - 33.3 vs 33.4**. ఒక ఖాళీ board మీద నడిచినా, పదహారు పాములు నిచ్చెనలు ఉన్న classic board మీద నడిచినా, సగటు ఒకటే.

కానీ ఆకారం పూర్తిగా వేరు:

- మధ్యస్థ 32 → **29** - సాధారణ ఆట చిన్నది అయింది
- p99 53 → **101** - చెడ్డ ఆట రెట్టేంపు అయింది
- గరిష్ఠం 84 → **250** - మూడు రెట్లు

అంటే ఆ board రూపకర్త **సగటుని కాదు, నాటకీయతని** design చేశాడు. చాలా ఆటలు త్వరగా ముగుస్తాయి; కొన్ని భయంకరంగా సాగుతాయి. **అదే ఆటని ఆసక్తికరంగా చేస్తుంది.**

మరియు నేను దీన్ని ఊహించలేదు - నేను classic board ఆటని పొడిగిస్తుందని అనుకున్నాను. కొలిచాక తెలిసింది.

మొదటిసారి 20,000 ఆటలు నడిపినప్పుడు రెండు సగటులూ **సరిగ్గా 33.3** వచ్చాయి, మరియు నేను "ఇవి ఖచ్చితంగా సమానం" అని రాయబోయాను. నాలుగు వేర్వేరు seeds తో పరీక్షిస్తే - ఖాళీ board 33.30-33.35, classic 33.38-33.71. అంటే classic **కొంచెం ఎక్కువ**, సమానం కాదు.

ఒక run నుంచి "సరిగ్గా సమానం" అని తీర్మానించడం - అది ఒక తప్పు, మరియు దాన్ని పట్టుకున్నది పలు seeds మాత్రమే.

మరియు §8 యొక్క ముగింపు నియమాలు:

classic board · వేర్వేరు "100 దాటేత" నియమాలు:

నియమం	సగటు	మధ్యస్థ	p90	p99	గరిష్ఠం
అక్కడే ఉండిపోవడం	33.4	29	58	101	250
వెనక్కి తిరగడం	36.7	30	67	121	242
దాటినా గెలుపే	30.3	26	53	90	229

"వెనక్కి తిరగడం" ఆటని 3 వంతులు పొడిగిస్తుంది - ఎందుకంటే అది ఆటగాడిని వెనక్కి నెట్టడమే కాదు, వెనక్కి వచ్చిన గడిలో ఒక పాము ఉండొచ్చు.

దశల నుంచి ఇక్కడికి - ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	<code>jumps</code> Map, వంతులు	మౌలిక అస్థిపంజరం
§4 (విరుపు)	<code>#resolved</code> - గొలుసు చివర	ఆటగాడు పాము నోట్లో నిలబడ్డాడు
§5	నిర్మాణ సమయంలో లెక్కించడం	ఆటలో lookup, లూప్ కాదు
§7 (విరుపు)	<code>finish</code> నియమం	103వ గడి
§8	మూడు నియమ classes	మూడూ నిజమైన ఆటల్లో వాడేవే

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§10 (విరుపు)	వలయ తనిఖీ	while అనంతంగా తిరిగింది
§11	అన్ని సమస్యలను సేకరించడం	Board file సరిచేసేవారికి
§12	Die బయటికి	పరీక్షించడానికి, అనుకరించడానికి
§13	—	ఆ అనుకరణే §13 యొక్క కొలతని సాధ్యం చేసింది

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

Code చిన్నది – సమయం అంతా నియమాల మీదే

7 నిమి	Clarify – గోలునులు? 100 దాటేతే? board ఎవరిస్తారు?
8 నిమి	మౌలిక ఆట · వేగంగా దాటండి
10 నిమి – గోలునులు	నిర్మాణ సమయంలో పరిష్కరించడం
12 నిమి – Validation	వలయం · ఇక్కడే మీరు నిలబడతారు
8 నిమి	Die · finish నియమాలు · అనుకరణ

ఏమి తప్పక చెప్పాలి

1. "నిచ్చిన చివర పాము ఉంటే?" అని మీరే అడగండి (§4) – ఇది మొదటి రెండు నిమిషాల్లో, మరియు ఇది ఆటని ఊహించుకుంటున్నారని చూపిస్తుంది.
2. గోలునుని నిర్మాణ సమయంలో పరిష్కరించడం (§5) – ఆటలో lookup. మరియు అదే వలయాలని బయటపెడుతుంది.
3. Board validation (§11) – ఇదే ఈ problem lo మీరు నిలబడే చోటు. చాలా మంది దీన్ని అస్సలు ప్రస్తావించరు. వలయం, పైకి వెళ్ళే పాము, నకిలీ start – మూడూ చెప్పండి.
4. "100 దాటేతే" ఒక మారే నియమం (§8) – మూడు నిజమైన ఎంపికలు ఉన్నాయని చెప్పండి.
5. Die ని inject చేయడం (§12) – "so the game is deterministic in tests and I can Monte-Carlo it."

ఏమి వదిలేయాలి

- Board రూపకల్పన (ఏ గడిలో ఏ పాము) – అది ఒక game-design ప్రశ్న, అడిగితేనే.
- UI, rendering – పూర్తిగా వదిలేయండి.
- BounceBack మరియు Any0vershootWins రాయెద్దు – interface చూపించి, ఒకటి రాయండి.
- అనుకరణ – ఒక వాక్యం: "with an injected die I could Monte-Carlo the board to check it's balanced." అది ఒక బలమైన ముగింపు.

15. నీటితో చెప్పాల్సిన English script

"Three questions before I start. If a ladder puts me on a square that has a snake's head, does the snake apply? What happens if I'm on ninety-seven and roll a six? And who supplies the board – can I assume it's valid?

The first one matters because it's the difference between an if and a while. The obvious implementation adds the die, looks up one jump, and stops. That leaves the player standing on a snake's head, which can't happen on a physical board – and worse, on their next turn they move off it, so that snake never fires at all. Landing on a square means the square's rule applies, and that's recursive.

I'd resolve those chains once, when the board is constructed, rather than every turn. Two reasons: during play it becomes a map lookup instead of a loop, and walking the chains at construction is exactly where cycles show up.

Because there's a third problem hiding in that fix. If the board has a ladder from five to twenty-five and a snake from twenty-five back to five, the while loop never terminates. That's not a crash with a stack trace – the game just stops, and only when a particular player rolls a particular number, so it won't show up in tests.

The code isn't wrong there; the data is. So I'd validate the board in the constructor and refuse to build an invalid one. Cycles, snakes that go up, ladders that go down, two jumps from the same square, jumps off the board, a jump from the winning square. And I'd collect all the problems rather than throwing on the first, because whoever is fixing a board file wants the whole list. This is one of the few places I'd actually throw rather than return a result object – a half-built board shouldn't exist.

On overshoot, there are three real rules people play: you must land exactly and a bust wastes the turn, you bounce back by the excess, or any overshoot wins. All three are legitimate, so that's a strategy object rather than an if. One ordering detail: apply the overshoot rule first and the board's jump resolution second, because bouncing back can land you on a snake.

Last, I'd inject the die rather than calling Math.random inside. That makes games deterministic in tests, and it means I can run the board through a Monte Carlo simulation to check it's balanced. I actually did that – fifty thousand games. The interesting result is that the classic board has almost the same mean length as a board with no snakes or ladders at all, about thirty-three turns either way. What it changes is the spread: the median drops from thirty-two to twenty-nine, but the ninety-ninth percentile goes from fifty-three to a hundred and one, and the worst game from eighty-four turns to two hundred and fifty. The board isn't designed to change how long a game takes – it's designed to add variance."

16. Follow-ups – బహుళ ఆటగాళ్ళు, ఆన్‌లైన్, board తయారీ

Follow-up	జవాబు	మార్గ classes
"6 వేస్తే మళ్ళీ వంతు"	<code>extraTurnOnSix</code> ఇప్పటికే ఉంది	0
"మొదలుపెట్టడానికి 6 కావాలి"	<code>play</code> lo ఒక షరతు (<code>from === 0 && die !== 6</code>)	1 పంక్తి
"రెండు పాచికలు"	<code>class TwoDice { roll() { ... } }</code>	+1 కొత్తది
"ఒక గడిలో ఇద్దరు ఉంటే వెనకవాడు మొదటికి"	<code>play</code> lo ఒక తనిఖీ	<code>Game</code>
"నలుగురు ఆటగాళ్ళు"	<code>players</code> ఇప్పటికే array	0
"ఒక సమతుల్యమైన board తయారు చెయ్యి"	కింద చూడండి	—
"ఆన్‌లైన్ – ఇద్దరు వేర్వేరు చీట్లు"	కింద చూడండి	Server layer

సమతుల్యమైన board తయారు చేయడం

"§13 యొక్క అనుకరణ ఇప్పుడు ఒక **సాధనం** అయింది. ఒక board ఇచ్చి, 50,000 ఆటలు నడిపి, దాని **ఆకారాన్ని** చూడొచ్చు.

ఒక board designer ఇలా పనిచేయొచ్చు: యాదృచ్ఛిక boards తయారు చేసి, ప్రతిదాన్నీ అనుకరించి, కావాల్సిన లక్షణాలు ఉన్నవాటిని ఎంచుకోవడం – ఉదాహరణకి "మధ్యస్థ 30 కంటే తక్కువ, p99 90 కంటే ఎక్కువ".

ఇది ఒక సాధారణ నమూనా: ఒక system ని పరీక్షించగలిగేలా చేస్తే (§12), అదే పరీక్షా సాధనం ఒక **design సాధనం** అవుతుంది."

ఆన్‌లైన్ ఆట

"పాచిక **server** lo వేయాలి – లేకపోతే client 6 అని చెప్పుకోవచ్చు. నా `Die` ఇప్పటికే inject చేయబడింది, కాబట్టి server తన సొంత RNG ఇస్తుంది.

మరియు `play(by)` ఇప్పటికే ఎవరు ఆడుతున్నారో తీసుకుని `NOT_YOUR_TURN` తిరస్కరిస్తుంది – **client ని నమ్మాల్సిన అవసరం లేదు.**

నవీకరణలు పంపడానికి Deep Dive 07 యొక్క event bus. **0 కొత్త concepts.**

మరియు `#history` ఉంది కాబట్టి – ఒక ఆటగాడు disconnect అయి తిరిగి వస్తే, మొత్తం ఆటని మళ్ళీ ఆడించి స్థితిని పునర్నిర్మించొచ్చు."

17. ఏమి నేర్చుకున్నాం

అలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
నియమం పునరావృత్తమా, ఒకసారా?	<code>if</code> vs <code>while</code> (§4)	Redirects, rewrites, aliases
నిర్మాణ సమయంలో లెక్కించడం	<code>#resolved</code> (§5)	Deep Dive 12 §5, 17 §9
తప్పు <code>code</code> లో కాదు, డేటాలో	వలయం (§10)	Configs, schemas, board/level files
అన్ని సమస్యలనీ ఒకేసారి చెప్పడం	<code>problems[]</code> (§11)	Form validation, compilers, linters
ఎప్పుడు <code>throw</code> , ఎప్పుడు <code>{ok:false}</code>	Constructor (§11)	సగం కట్టిన వస్తువు ఉండకూడదు
బయటి ప్రపంచాన్ని <code>inject</code> చెయ్యి	<code>Die</code> (§12)	Deep Dive 14 §12 · clocks, RNG, IO
పరీక్షా సాధనం = <code>design</code> సాధనం	50,000 ఆటలు (§13, §16)	Load tests, simulations, fuzzers

ఒక చివరి అలోచన – నేను ఈ **PROBLEM** ని తక్కువ అంచనా వేశాను

ఈ doc మొదట్లో చెప్పాను: నేను Snake & Ladder ని "పలచని problem" అని అనుకున్నాను. వ్యాహం లేని ఆటలో design ఏముంటుంది?

మూడు విరుపులు దొరికాయి, మరియు అవి మూడూ **వేర్వేరు రకాలు**: ఒకటి నియమాన్ని తప్పుగా అర్థం చేసుకోవడం (§4), ఒకటి ఒక నిర్ణయాన్ని దాచేయడం (§7), ఒకటి డేటాని నమ్మడం (§10). మరియు చివరికి ఒక కొలత నా అంచనాని తలకిందులు చేసింది (§13).

పాఠం: "ఈ problem లో ఏమీ లేదు" అనేది ఒక ఊహ, ఒక వాస్తవం కాదు. మరియు ఏ ఊహనైనా పరీక్షించే మార్గం ఒక్కటే – దాన్ని రాసి చూడటం.

Interview లో ఇది మరింత నిజం: "సులభమైన" problem ఇచ్చారంటే, వారు చూసేది పరిష్కారం కాదు – మీరు ఎంత లోతుకి వెళ్తారో.

ఇక్కడి నుంచి ఎక్కడికి

ఈ series లో ఇప్పటివరకు: **01 Parking Lot · 02 Cache · 03 Rate Limiter · 04 BookMyShow · 05 Splitwise · 06 Elevator · 07 Pub-Sub · 08 HashMap · 09 Chess · 10 Meeting Scheduler · 11 Food Delivery · 12 File System · 13 Leaderboard · 14 Text Editor · 15 Library · 16 Tic-Tac-Toe · 17 Autocomplete · 18 Snake & Ladder.**

Deep Dive 16 (Tic-Tac-Toe) దీనికి జంట – అక్కడా ఒక ఆట, అక్కడా నియమాలని బయటికి తీయడం. తేడా: అక్కడ ఆటగాళ్ళు నిర్ణయాలు తీసుకుంటారు.

Snake & Ladder – అడుగు అడుగునా · ఈ doc లో ఉన్న ప్రతి output, 50,000 ఆటల కొలతతో సహా నిజంగా `node` లో run చేసి తీసినదే 